

```
# Advanced Econometrics - Group Assignment 1
```

```
# Group 18:
```

```
# Phuc Nguyen, 2779150
```

```
# Tu Nguyen, 2849240
```

```
# Boris de Buck, 2732664
```

```
# Anna van Dam, 2861128
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from scipy.special import gammaln
```

```
from scipy.optimize import minimize
```

```
# Load data and sanity check
```

```
def load_returns(filepath="data/return_data.csv"):
```

```
    df = pd.read_csv(filepath, parse_dates=["DlyCalDt"])
```

```
    # Scale returns to percentage points
```

```
    df["DlyRet"] = df["DlyRet"] * 100
```

```
    # Reshape: one column per ticker, indexed by date
```

```
    wide = df.pivot(index="DlyCalDt", columns="Ticker", values="DlyRet")
```

```
    wide = wide.sort_index()
```

```
    return wide
```

```
def sanity_check(data):
```

```
    """
```

```
    Check AAPL stats against assignment's reference values.
```

```
    """
```

```
    aapl = data["AAPL"]
```

```
    assert abs(aapl.mean() - 0.1071) < 0.001, f"Mean mismatch: {aapl.mean():.4f}"
```

```
    assert abs(aapl.std() - 1.7832) < 0.001, f"Std mismatch: {aapl.std():.4f}"
```

```
    assert abs(aapl.min() - (-12.8647)) < 0.001, f"Min mismatch: {aapl.min():.4f}"
```

```
    assert abs(aapl.max() - 11.9808) < 0.001, f"Max mismatch: {aapl.max():.4f}"
```

```
    print("AAPL sanity check passed.")
```

```
# Question 2
```

```
def news_impact(x, alpha, delta, gamma):
```

```
    """
```

```
    News-impact curve: isolates how a shock  $x_{t-1}$  affects volatility,  
    dropping terms that don't depend on  $x_{t-1}$ .
```

```
    From  $\sigma_t^2 = \omega + (\alpha + \delta \tanh(-\gamma x)) * ((x - \mu - \lambda \sigma^2) / \sigma)^2 + \beta \sigma^2$ 
```

```
    - set  $\mu=0$ ,  $\lambda=0$ ,  $\sigma_{t-1}=1$  -> squared term reduces to  $x^2$ 
```

```
    - drop  $\omega$  (constant, unrelated to  $x$ ) and  $\beta \sigma^2$  (past  
    volatility's own persistence, not the news effect)
```

```
    What remains is purely the shock-driven piece:
```

```
    
$$\text{NIC}(x) = (\alpha + \delta \tanh(-\gamma x)) * x^2$$

```

```
    """
```

```
    return (alpha + delta * np.tanh(-gamma * x)) * x**2
```

```
def plot_news_impact_curves():
```

```
    alpha = 0.4
```

```
    gammas = [0.01, 0.1, 1]
```

```

deltas = [0.3, 0.1, 0, -0.3]
x = np.linspace(-5, 5, 500)

fig, axes = plt.subplots(2, 2, figsize=(10, 8), sharex=True, sharey=True)
axes = axes.flatten()

for i, delta in enumerate(deltas):
    ax = axes[i]
    for gamma in gammas:
        y = news_impact(x, alpha, delta, gamma)
        ax.plot(x, y, label=f" $\gamma={\gamma}$ ")
    ax.set_title(f" $\delta = {\delta}$ ")
    ax.set_xlabel(r" $x_{t-1}$ ")
    ax.set_ylabel("News impact")
    ax.legend()
    ax.grid(alpha=0.3)

fig.suptitle("News-Impact Curves for Different  $\delta$  and  $\gamma$  ( $\alpha=0.4$ )")
plt.tight_layout()
plt.savefig("outputs/news_impact_curves.png", dpi=150)
plt.show()

```

Question 3

```

def descriptive_stats(data):
    """
    Compute descriptive statistics for each ticker in the dataset.
    Return a DataFrame with ticker names as index and statistics as columns.
    """
    stats_dict = {}
    for ticker in data.columns:
        series = data[ticker].dropna()
        stats_dict[ticker] = {
            "Obs": len(series),
            "Mean": series.mean(),
            "Median": series.median(),
            "Std Dev": series.std(),
            "Skewness": series.skew(),
            "Excess Kurtosis": series.kurtosis(),
            "Min": series.min(),
            "Max": series.max(),
        }
    return pd.DataFrame(stats_dict).T

def plot_returns(data):
    fig, axes = plt.subplots(2, 2, figsize=(12, 8), sharex=True)
    axes = axes.flatten()

    for i, ticker in enumerate(data.columns):
        axes[i].plot(data.index, data[ticker], linewidth=0.5)
        axes[i].set_title(ticker)
        axes[i].set_ylabel("Return (%)")
        axes[i].grid(alpha=0.3)

    fig.suptitle("Daily Returns, 2011-2023")
    plt.tight_layout()
    plt.savefig("outputs/returns_panels.png", dpi=150)
    plt.show()

```

```

# Question 4
def log_t_pdf(z, nu):
    """Log density of the standard Student-t at value z."""
    return (
        gammaln((nu + 1) / 2)
        - gammaln(nu / 2)
        - 0.5 * np.log(nu * np.pi)
        - ((nu + 1) / 2) * np.log1p(z**2 / nu) # using log1p for numerical stability
    )

def unpack_params(theta, model):
    """Map flat theta to named params, fixing lambda/delta/gamma=0 for nested models."""
    if model == "GARCH":
        mu, omega, alpha, beta, nu = theta
        lam, delta, gamma = 0.0, 0.0, 0.0
    elif model == "GARCH-M":
        mu, lam, omega, alpha, beta, nu = theta
        delta, gamma = 0.0, 0.0
    elif model == "GARCH-M-L":
        mu, lam, omega, alpha, beta, delta, gamma, nu = theta
    else:
        raise ValueError(model)
    return mu, lam, omega, alpha, beta, delta, gamma, nu

def filter_sigma2(x, mu, lam, omega, alpha, beta, delta, gamma, sigma2_1):
    """Run the GARCH-M-L recursion. Returns the sigma2 path, or None if the
    recursion hits a non-positive/exploded variance (signals an invalid
    parameter region during optimization)."""
    T = len(x)
    sigma2 = np.empty(T)
    sigma2[0] = sigma2_1

    for t in range(T - 1):
        z = (x[t] - mu - lam * sigma2[t]) / np.sqrt(sigma2[t])
        coef = alpha + delta * np.tanh(-gamma * x[t])
        sigma2[t + 1] = omega + coef * z**2 + beta * sigma2[t]
        if not np.isfinite(sigma2[t + 1]) or sigma2[t + 1] <= 0 or sigma2[t + 1] > 1e8:
            return None

    return sigma2

def log_likelihood(theta, x, sigma2_1, model):
    """
    Ordinary (unpenalized) total log-likelihood -- used for reporting
    LogLik, AIC and BIC in the results table, and for the AAPL Table 1
    validation check.
    """
    mu, lam, omega, alpha, beta, delta, gamma, nu = unpack_params(theta, model)

    # Parameter restrictions from the assignment
    if beta < 0 or nu <= 2:
        return -np.inf

    # Assignment restriction: alpha > |delta|
    # For GARCH and GARCH-M, delta = 0, so this also requires alpha > 0.
    if alpha <= abs(delta):

```

```

        return -np.inf

# gamma > 0 only applies to the GARCH-M-L model.
# In GARCH and GARCH-M, gamma is fixed at zero by definition.
if model == "GARCH-M-L" and gamma <= 0:
    return -np.inf

sigma2 = filter_sigma2(x, mu, lam, omega, alpha, beta, delta, gamma, sigma2_1)

if sigma2 is None:
    return -np.inf

z = (x - mu - lam * sigma2) / np.sqrt(sigma2)
ll = np.sum(log_t_pdf(z, nu) - 0.5 * np.log(sigma2))

if not np.isfinite(ll):
    return -np.inf

return ll

def neg_penalized_log_likelihood(theta, x, sigma2_1, model):
    """
    Objective function MINIMIZED during optimization -- NOT the same as
    what gets reported. For GARCH-M-L, applies the assignment's penalty
    (-0.001*gamma^2) to prevent gamma from wandering when delta ~ 0.
    Reporting always uses the plain log_likelihood() above, evaluated at
    the resulting estimate.
    """
    mu, lam, omega, alpha, beta, delta, gamma, nu = unpack_params(theta, model)

    ll = log_likelihood(theta, x, sigma2_1, model)
    if not np.isfinite(ll):
        return 1e10

    if model == "GARCH-M-L":
        ll -= 0.001 * gamma**2

    return -ll

# Initial values for estimation
def get_sigma2_1(x):
    """sigma_1^2 = variance of first 50 obs, ddof=0 (divide by 50, not 49) per spec."""
    return np.var(x[:50], ddof=0)

def get_initial_values(x, model, T=2500):
    s2 = np.var(x[:T], ddof=0)
    if model == "GARCH":
        return np.array([0.0, s2 / 50, 0.05, 0.9, 10.0])
    elif model == "GARCH-M":
        return np.array([0.0, 0.0, s2 / 50, 0.05, 0.9, 10.0])
    elif model == "GARCH-M-L":
        return np.array([0.0, 0.0, s2 / 50, 0.05, 0.9, 0.01, 0.01, 10.0])

PARAM_NAMES = {
    "GARCH": ["mu", "omega", "alpha", "beta", "nu"],
    "GARCH-M": ["mu", "lambda", "omega", "alpha", "beta", "nu"],

```

```

    "GARCH-M-L": ["mu", "lambda", "omega", "alpha", "beta", "delta", "gamma", "nu"],
}

# Widened eps and relaxed gtol vs. BFGS defaults: the default finite-difference
# step is too small for a 2,500-step recursive likelihood, causing "precision
# loss" warnings even at the numerical optimum.
BFGS_OPTIONS = {"maxiter": 5000, "gtol": 1e-3, "eps": 1e-6}

# The estimation loop
def estimate_all(data, T=2500):
    results = []

    for ticker in data.columns:
        series = data[ticker].dropna().values
        x = series[:T]
        sigma2_1 = get_sigma2_1(x)

        for model in ["GARCH", "GARCH-M", "GARCH-M-L"]:
            theta0 = get_initial_values(x, model, T)
            # First try BFGS, as recommended in the assignment.
            res = minimize(
                neg_penalized_log_likelihood,
                theta0,
                args=(x, sigma2_1, model),
                method="BFGS",
                options=BFGS_OPTIONS,
            )
            # IF BFGS does not converge, retry from the original starting values
            # using Powell, which is more robust to the parameter penalties
            if not res.success:
                print(
                    f"WARNING: {ticker} {model} BFGS did not converge: "
                    f"{res.message}. Retrying with Powell."
                )

                res_powell = minimize(
                    neg_penalized_log_likelihood,
                    theta0,
                    args=(x, sigma2_1, model),
                    method="Powell",
                    options={"maxiter": 3000, "xtol": 1e-7, "ftol": 1e-7},
                )
                # Use Powell results if it achieved a lower objective value
                # or a better objective value than the failed BFGS result
                if res_powell.success and res_powell.fun <= res.fun + 1e-4:
                    res = res_powell

            # Final convergence message
            if not res.success:
                print(
                    f"WARNING: {ticker} {model} still did not converge: "
                    f"{res.message}"
                )

        k = len(theta0)
        # important:
        # res.fun contains the PENALISED objective for GARCH-M-L
        # For reporting LogLik, AIC, BIC, we need the unpenalized likelihood
        loglik = log_likelihood(res.x, x, sigma2_1, model)

```

```

aic = 2 * k - 2 * loglik
bic = k * np.log(T) - 2 * loglik

row = {
    "Ticker": ticker,
    "Model": model,
    "LogLik": loglik,
    "AIC": aic,
    "BIC": bic,
    "Converged": res.success,
}
for name, val in zip(PARAM_NAMES[model], res.x):
    row[name] = val
results.append(row)

return pd.DataFrame(results)

# Question 5
def plot_q5(data, results, T_in=2500):
    tickers = [t for t in data.columns if t != "AAPL"]
    x_grid = np.linspace(-5, 5, 500)
    fig, axes = plt.subplots(len(tickers), 2, figsize=(12, 12))

    for row, ticker in enumerate(tickers):
        series = data[ticker].dropna()
        x_full = series.values
        dates = series.index

        # ---- Left panel: news-impact curves for all three models ----
        ax_left = axes[row, 0]
        for model in ["GARCH", "GARCH-M", "GARCH-M-L"]:
            p = results[
                (results["Ticker"] == ticker) & (results["Model"] == model)
            ].iloc[0]
            alpha, delta, gamma = p["alpha"], p.get("delta", 0.0), p.get("gamma", 0.0)
            if pd.isna(delta):
                delta = 0.0
            if pd.isna(gamma):
                gamma = 0.0
            y = news_impact(x_grid, alpha, delta, gamma)
            ax_left.plot(x_grid, y, label=model)
        ax_left.set_title(f"News impact curves for {ticker}")
        ax_left.set_xlabel(r" $x_{t-1}$ ")
        ax_left.set_ylabel("News impact")
        ax_left.legend()
        ax_left.grid(alpha=0.3)

        # ---- Right panel: filtered volatility (GARCH-M-L) over full sample ----
        ax_right = axes[row, 1]
        p = results[
            (results["Ticker"] == ticker) & (results["Model"] == "GARCH-M-L")
        ].iloc[0]
        sigma2_1 = get_sigma2_1(x_full[:T_in])
        sigma2_full = filter_sigma2(
            x_full,
            p["mu"],
            p["lambda"],
            p["omega"],
            p["alpha"],

```

```

        p["beta"],
        p["delta"],
        p["gamma"],
        sigma2_1,
    )
    sigma_full = np.sqrt(sigma2_full)

    ax_right.plot(dates, x_full, color="lightgray", linewidth=0.5, label="Returns")
    ax_right.plot(dates, sigma_full, color="C0", linewidth=1, label="GARCH-M-L")
    ax_right.axvline(dates[T_in], color="black", linestyle="--", linewidth=1)
    ax_right.set_title(f"Filtered volatilities for {ticker}")
    ax_right.legend()
    ax_right.grid(alpha=0.3)

plt.tight_layout()
plt.subplots_adjust(hspace=0.4)
plt.savefig("outputs/q5_panels.png", dpi=150)
plt.show()

# Question 6: Value-at-Risk estimates based on simulation
### Step 1
def get_theta_from_results(results, ticker, model):
    """
    Extract the estimated parameter vector for one stock and model
    in the same order used during estimation.
    """
    row = results[(results["Ticker"] == ticker) & (results["Model"] == model)].iloc[0]

    theta = np.array([row[name] for name in PARAM_NAMES[model]], dtype=float)

    return theta

def get_state_at_forecast_origin(
    data, results, ticker, model, forecast_date="2021-01-04"
):
    """
    Filter the model using observed returns up to and including
    the forecast origin.

    Parameters remain fixed at their Q4 estimates based on the
    first 2,500 observations.
    """
    forecast_date = pd.Timestamp(forecast_date)

    # Observed returns available at the forecast origin
    series = data[ticker].dropna()
    history = series.loc[:forecast_date]

    if len(history) == 0 or history.index[-1] != forecast_date:
        # forecast_date before series starts or forecast date not presented
        # in index (not a trading day)
        raise ValueError(
            f"Forecast date {forecast_date.date()} not found for {ticker}."
        )

    x_history = history.values # all past returns up to forecast points

    # Q4 parameter estimates: NO re-estimation

```

```

theta = get_theta_from_results(results, ticker, model)

mu, lam, omega, alpha, beta, delta, gamma, nu = unpack_params(theta, model)

# Same initial variance convention as in Q4
sigma2_1 = get_sigma2_1(x_history)

# Filter through all observed returns up to Jan 4, 2021
sigma2_history = filter_sigma2(
    x_history, mu, lam, omega, alpha, beta, delta, gamma, sigma2_1
)

if sigma2_history is None:
    raise ValueError(f"Invalid filtered variance path for {ticker} {model}.")

# State at t0 = January 4, 2021
x_t0 = x_history[-1]
sigma2_t0 = sigma2_history[-1]

return {
    "theta": theta,
    "x_t0": x_t0,
    "sigma2_t0": sigma2_t0,
    "nu": nu,
    "n_observations": len(x_history),
    "forecast_date": forecast_date,
}

### Step 2: One-step volatility update and first simulated future return
def update_sigma2_one_step(x_t, sigma2_t, mu, lam, omega, alpha, beta, delta, gamma):
    """
    Compute  $\sigma^2_{t+1}$  from the current return  $x_t$ 
    and current variance  $\sigma^2_t$ .
    """

    sigma_t = np.sqrt(sigma2_t)

    # Standardized innovation at time t
    z_t = (x_t - mu - lam * sigma2_t) / sigma_t

    # Shock coefficient including leverage effect
    coef = alpha + delta * np.tanh(-gamma * x_t)

    # GARCH-M-L variance recursion
    sigma2_next = omega + coef * z_t**2 + beta * sigma2_t

    # Safety check
    if not np.isfinite(sigma2_next) or sigma2_next <= 0:
        raise ValueError("Invalid simulated variance encountered.")

    return sigma2_next

def simulate_next_return(sigma2_next, mu, lam, nu, rng):
    """
    Simulate  $x_{t+1}$  conditional on  $\sigma^2_{t+1}$ .
    """

    # Draw innovation from Student-t distribution
    epsilon_next = rng.standard_t(df=nu)

```

```

# Generate next-period return
x_next = mu + lam * sigma2_next + np.sqrt(sigma2_next) * epsilon_next

return x_next, epsilon_next

### Step 3: Simulate one complete future return path
def simulate_return_path(state, model, horizon, rng):
    """
    Simulate one future return path from t0+1 up to t0+horizon.

    The simulation starts from the filtered state at the forecast origin
    and recursively updates volatility using each simulated return.
    """

    # Extract estimated parameters
    mu, lam, omega, alpha, beta, delta, gamma, nu = unpack_params(
        state["theta"],
        model,
    )

    # Starting point: observed state on January 4, 2021
    x_current = state["x_t0"]
    sigma2_current = state["sigma2_t0"]

    # Storage for simulated future returns and variances
    simulated_returns = np.empty(horizon)
    simulated_sigma2 = np.empty(horizon)

    for h in range(horizon):
        # Step 1: update variance using current x_t and sigma^2_t
        sigma2_next = update_sigma2_one_step(
            x_current, sigma2_current, mu, lam, omega, alpha, beta, delta, gamma
        )

        # Step 2: draw innovation and simulate x_{t+1}
        x_next, epsilon_next = simulate_next_return(sigma2_next, mu, lam, nu, rng)

        # Store simulated future values
        simulated_returns[h] = x_next
        simulated_sigma2[h] = sigma2_next

        # Move forward one period
        x_current = x_next
        sigma2_current = sigma2_next

    return simulated_returns, simulated_sigma2

### Step 4: Simulate many paths and construct compound returns
def compound_return(simulated_returns, horizon):
    """
    Compute the compound holding-period return over a given horizon.

    Returns are stored in percentage points, so divide by 100
    before compounding and multiply by 100 afterwards.
    """

    return 100 * (np.prod(1 + simulated_returns[:horizon] / 100) - 1)

```

```

def simulate_compound_returns(
    state, model, n_simulations=10000, horizons=(1, 5, 20), seed=12345
):
    """
    Simulate many future paths and calculate the compound
    returns for the requested horizons.
    """

    max_horizon = max(horizons)

    # One RNG for the complete Monte Carlo experiment
    rng = np.random.default_rng(seed)

    # Rows = simulations
    # Columns = requested horizons
    compound_returns = np.empty((n_simulations, len(horizons)))

    for s in range(n_simulations):
        # Simulate one complete path up to the longest horizon
        sim_returns, _ = simulate_return_path(
            state,
            model,
            horizon=max_horizon,
            rng=rng,
        )

        # Use the same path for 1-, 5-, and 20-day returns
        for j, horizon in enumerate(horizons):
            compound_returns[s, j] = compound_return(
                sim_returns,
                horizon,
            )

    return compound_returns

### Step 5: VaR: Compute VaR from simulated compound returns
def calculate_var(
    compound_returns,
    horizons=(1, 5, 20),
    levels=(0.01, 0.05, 0.10),
):
    """
    Compute empirical VaR estimates from simulated
    compound-return distributions.

    Returns a DataFrame with horizons as rows and
    VaR probability levels as columns.
    """

    var_results = {}

    for j, horizon in enumerate(horizons):
        var_results[horizon] = {}

        for level in levels:
            var_results[horizon][level] = np.quantile(
                compound_returns[:, j],

```

```

        level,
    )

    var_df = pd.DataFrame(var_results).T

    var_df.index.name = "Horizon"
    var_df.columns = [f"{int(level * 100)}%" for level in levels]

    return var_df

### Step 6: Compute final VaR table for all stocks and models
def compute_var_table(
    data,
    results,
    tickers=("PFE", "JNJ", "MRK"),
    models=("GARCH", "GARCH-M", "GARCH-M-L"),
    n_simulations=10000,
    seed=12345,
):
    """
    Compute 1%, 5%, and 10% VaR for 1-, 5-, and 20-day
    compound returns for all three pharmaceutical stocks
    and all three models.
    """

    horizons = (1, 5, 20)
    levels = (0.01, 0.05, 0.10)

    model_labels = {
        "GARCH": "M1",
        "GARCH-M": "M2",
        "GARCH-M-L": "M3",
    }

    rows = []

    # Counter gives every stock/model combination a different,
    # but reproducible, random-number stream.
    simulation_counter = 0

    for ticker in tickers:
        for model in models:
            # State at January 4, 2021 using fixed Q4 estimates
            state = get_state_at_forecast_origin(
                data,
                results,
                ticker=ticker,
                model=model,
            )

            # Simulate future compound returns
            compound_simulations = simulate_compound_returns(
                state,
                model=model,
                n_simulations=n_simulations,
                horizons=horizons,
                seed=seed + simulation_counter,
            )

```

```

        # Calculate empirical VaR quantiles
        var_df = calculate_var(
            compound_simulations,
            horizons=horizons,
            levels=levels,
        )

        row = {
            "Stock": ticker,
            "Model": model_labels[model],
        }

        # Store 9 VaRs for this stock/model pair
        for horizon in horizons:
            for level in levels:
                level_name = f"{int(level * 100)}%"

                row[f"{horizon}d {level_name}"] = var_df.loc[
                    horizon,
                    level_name,
                ]

            rows.append(row)

        simulation_counter += 1

    return pd.DataFrame(rows)

def print_var_table(var_results):
    """Print and save the final Q6 VaR results table."""
    print("\n--- Q6 Final VaR Results ---")
    print(var_results.round(2).to_string(index=False))
    var_results.round(2).to_csv("outputs/q6_var_results.csv", index=False)

def validate_aapl_var(data, results, n_simulations=10000, seed=12345):
    """Check simulated AAPL VaR against the assignment's Table 2 reference."""
    print("\n--- Q6 AAPL validation against Table 2 ---")

    for model in ["GARCH", "GARCH-M", "GARCH-M-L"]:
        state = get_state_at_forecast_origin(data, results, ticker="AAPL", model=model)
        compound_simulations = simulate_compound_returns(
            state,
            model=model,
            n_simulations=n_simulations,
            horizons=(1, 5, 20),
            seed=seed,
        )
        var_check = calculate_var(
            compound_simulations, horizons=(1, 5, 20), levels=(0.01, 0.05, 0.10)
        )
        print(f"\n{model}")
        print(var_check.round(2))

if __name__ == "__main__":
    data = load_returns()
    print(data.head())
    print(f"\nShape: {data.shape}")

```

```

print(f"Tickers: {list(data.columns)}")
print(f>Date range: {data.index.min()} to {data.index.max()}\n")

sanity_check(data)
plot_news_impact_curves()

stats_df = descriptive_stats(data)
print(stats_df.round(4))
stats_df.round(4).to_csv("outputs/descriptive_stats.csv")
plot_returns(data)

results = estimate_all(data)
cols = [
    "Ticker",
    "Model",
    "mu",
    "lambda",
    "omega",
    "alpha",
    "beta",
    "delta",
    "gamma",
    "nu",
    "LogLik",
    "AIC",
    "BIC",
    "Converged",
]
results = results[cols]
print(results.round(4).to_string(index=False))
results.round(4).to_csv("outputs/garch_estimates.csv", index=False)

aapl = results[results["Ticker"] == "AAPL"]
print("\n--- AAPL check against Table 1 ---")
print("Expected LogLik  $\approx$  -4662 (M1), -4662 (M2), -4632 (M3)")
print(aapl[["Model", "LogLik"]])

plot_q5(data, results)

# Question 6
var_results = compute_var_table(
    data,
    results,
    tickers=("PFE", "JNJ", "MRK"),
    models=("GARCH", "GARCH-M", "GARCH-M-L"),
    n_simulations=10000,
    seed=12345,
)
print_var_table(var_results)
validate_aapl_var(data, results)

```